

Locality Sensitive Hashing Algorithm for MinHash Signatures

April 2, 2026

1 Introduction

So far we have worked on MinHashing algorithms and LSH scheme based on (p1, p2, d1, d2) probabilistic guarantees to distinguish between similar vs dissimilar items. This is a new algorithm to integrate into a Vector Search System because it has a well-known method of inserting and querying MinHash signatures into a HashTable data structure with a proof on collision probability based on a given threshold t (user-defined Jaccard similarity). The LSH technique developed here is a family of MinHash functions that can be combined (by the banding technique) to distinguish strongly between pairs at a low distance from pairs at a high distance. This technique is more likely to make close pairs be candidate pairs than distant pairs. The steepness of the S-curve in Figures below reflects how effectively we can avoid false positives and false negatives among the candidate pairs.

For example, this method works for retrieving feature vectors given a threshold similarity. For example, I want to get all feature vectors that are $t \geq 80\%$ or more similar; I am not interested in obtaining feature vectors that are less similar than the threshold. This method is adopted from “Chapter 3: FINDING SIMILAR ITEMS of Mining of Massive Datasets book”.

1.1 Locality Sensitive Functions

Let $d1 < d2$ be two distances according to Jaccard distance measure d . A family F of functions is said to be $(d1, d2, p1, p2)$ -sensitive if for every f in F :

- if $d(p, q) \leq d1$, then $\Pr(h(p) = h(q)) \geq p1$
- if $d(p, q) \geq d2$, then $\Pr(h(p) = h(q)) \leq p2$

where $p1 > p2$.

A MinHash function h makes x and y a candidate pair if and only if $h(x) = h(y)$. The family of minhash functions is a $(d1, d2, 1-d1, 1-d2)$ -sensitive family for any $d1$ and $d2$, where $0 \leq d1 < d2 \leq 1$. The reason is that if $d(x, y) \leq d1$, where d is the Jaccard distance, then $\text{SIM}(x, y) = 1 - d(x, y) \geq 1 - d1$. But

we know that the Jaccard similarity of x and y is equal to the probability that a minhash function will hash x and y to the same value. A similar argument applies to d_2 or any distance.

Example 1. We could let $d_1 = 0.3$ and $d_2 = 0.6$. Then we can assert that the family of minhash functions is a $(0.3, 0.6, 0.7, 0.4)$ -sensitive family. That is, if the Jaccard distance between x and y is at most 0.3 (i.e., $\text{SIM}(x, y) \geq 0.7$) then there is at least a 0.7 chance that a minhash function will send x and y to the same value, and if the Jaccard distance between x and y is at least 0.6 (i.e., $\text{SIM}(x, y) \leq 0.4$), then there is at most a 0.4 chance that x and y will be sent to the same value. Note that we could make the same assertion with another choice of d_1 and d_2 ; only $d_1 < d_2$ is required.

1.2 Motivation

Let us assume that a query vector q has similarity s to an input vector p , that is, (Jaccard) $\text{sim}(q, p) = s$. The probability that the MinHash value of q will be the same as the MinHash value of p is equal to s . This is true by the definition of MinHashing. If we generate five independent MinHash values for q , the probability of collision with the input MinHash value is s^r . Hash table collision occurs when the MinHash signatures of p and q are exactly the same. As an example, if s is 0.8, $s^5 = 0.33$. If the signature length is 3, then the collision probability is $s^3 = 0.51$. Therefore, to increase the probability of collision, it is better to consider a subset of the signature for hashing into the hash table.

2 AND, OR Amplification of Locality-Sensitive Family

Let us say that we break the MinHash signature into two chunks. For example, the signature (12, 9, 8, 25, 37) is divided into two chunks (12, 9, 8) and (25, 37). We can separately hash the two chunks into two hash tables independently. If we get a hash collision for at least one of the signature chunks for q , we will be able to retrieve the 80% similar input p . Taking $s=0.8$ as in the earlier example, the probability that the two MinHash chunks are the same in the first chunk is $(.8)^2 = 0.64$. The probability of not being the same is $1-0.64 = 0.36$. The probability that the two MinHash codes are the same in the second chunk is $(.8)^3 = 0.512$. Then, the probability of not being the same is $1-0.512 = 0.488$. The probability of no collision for both signature chunks = $0.36*0.488 = 0.176$. The probability that a collision occurs in at least one chunk = $1 - 0.176 = 0.82$.

This MinHash family built using the AND construction for each chunk and the OR construction using the two chunks is a recipe to increase the overall collision probability.

Definition 1 (AND Construction). *Suppose we are given a (d_1, d_2, p_1, p_2) -sensitive family F . We can construct a new family F' by the AND-construction on F . Each member of F' consists of r members of F for some fixed r . This is*

MinHash fm	P ₁	P ₂	Polygon P ₃	P ₄	...	P _N	
h ₁ (l)	•	5	41	7	---	---	band 1
h ₂ (l)	•	10	13	10	---	---	
h ₃ (l)	•	25	25	99	---	---	
h ₄ (l)			101				band 2
h ₅ (l)			48				
h ₆ (l)			69				
h ₇ (l)			31				band 3
h ₈ (l)			55				
h ₉ (l)			44				
h ₁₀ (l)			19				band 4
h ₁₁ (l)			91				
h ₁₂ (l)			101				

Fig. MinHash
Signature
Matrix

$$\text{Collision Probability} = 1 - (1 - s^r)^b$$

$s = \text{Jaccard Similarity Poly}$
 $r = \text{rows in a band}$
 $b = \text{bands}$

$$t \approx \left(\frac{1}{b}\right)^{\frac{1}{r}}$$

$t = \text{threshold similarity}$

Figure 1: a) Partitioning the signature matrix into $b=4$ bands where each band has $r=3$ rows. $b*r$ = signature length = 12. MinHash signature for a feature vector consists of 12 integers (single column). Partial signature of a Feature Vector 3 (P_3) in the first band is (41,13,25).

We use $b = 4$ hash tables (one table for each band) to implement insertion and querying of a feature vector by splitting its signature into 4 parts. Each hash table insertion can happen independently. We evaluate hash function for Hash Table HT1 with input (41,13,25) to insert it into Hash Table 1.

Similarly, Partial signature of Feature Vector 3 in band 2 is (101, 48, 69). So, for band 2, we insert (101, 48, 69) into another Hash table HT2. The hash function uses dot product with a random vector modulo Hash Table size to index into the hash table.

b) $s = \text{Jaccard similarity}$. $t = \text{user-defined similarity threshold}$. Given two feature vectors (a,b) with Jaccard similarity s , the formula $1 - (1 - s^r)^b$ gives the probability that the pair (a,b) will become a candidate pair.

analogous to concatenating r MinHash codes (a subset of signature in a band) for hashing. Since the members of F are independently chosen to make a member of F' , we can assert that F' is a $(d1, d2, (p1)^r, (p2)^r)$ -sensitive family. For any p , if p is the probability that a member of F will declare (x, y) to be a candidate pair, then the probability that a member of F will so declare is p^r . AND construction decreases the probability for any p to p^r .

Definition 2 (OR Construction). *If we choose to split the signature matrix into b partitions (bands), then we can invoke OR construction in the family of hash functions F . The OR-construction turns a $(d1, d2, p1, p2)$ -sensitive family F into a $(d1, d2, 1 - (1 - p1)^b, 1 - (1 - p2)^b)$ -sensitive family F' . The OR-construction mirrors the effect of combining several bands: x and y become a candidate pair if any band makes them a candidate pair.*

We see that when we consider the full signature length for hashing, and the collision probability is 0.33. However, the collision probability drastically improves by the AND followed by OR construction of the new family of hash functions to 0.82.

The LSH theory is a motivation to pursue the splitting of MinHash signatures in a Vector Search System. By implementing the methodology presented below, we can study the trade-offs between recall, precision, run-time, false positives, and false negatives. In what follows, I will describe how LSH's S-curve can minimize both false positives and false negatives. In addition, we will connect this theory to the similarity threshold t .

3 Partitioning Signatures for multiple hashing

Signature matrix: Assume that we have generated a signature matrix using MinHash algorithm for N feature vectors and we have generated 12 minhashes. Therefore, a signature is 12 integers long and the dimension of the signature matrix is $N \times 12$. The 12 MinHash functions are the rows in the table and the columns comprise N feature vectors (or documents).

Matrix partitioning: Following the technique from Chapter 3, let us partition the signature matrix into b bands where each band has r rows. In our signature table, we show 4 bands where each band has 3 rows. Therefore, $b = 4$ and $r = 3$. $b \times r = \text{signature length} = 12$. Figure 1 illustrates how to partition the signature matrix into multiple bands in order to multi-hash the signatures into multiple hashtables.

Algorithm 1 shows the multi-hash table approach to partition a signature into b chunks and then separately hash each chunk to a different hash table. If we implement Algorithm 1 and integrate it into a Vector Search system, we anticipate higher recall and reduction of false positives. The algorithm can be tuned to improve the performance of LSH based vector similarity search by adjusting a single parameter (#bands). Figure 3 shows the probability of finding candidates given different choices of signature partition by varying b

and r . Figure 4 shows the probability of being a candidate given the similarity between two feature vectors.

Following the LSH scheme, given a query signature vector, Algorithm 2 shows how to perform a query using multiple hashtables. Given a query, if the query signature chunk matches in any one of the bands, we get a candidate for further refinement. There is a high likelihood that such a partial signature match will happen in at least one of the bands of the signature matrix for similar feature vectors because they have similar signatures.

Algorithm 1 Insert a signature of a feature vector into b Hash Tables

Require: 1. Signature Vector $M = (M_1, M_2, \dots, M_L)$ of length L of Vector.

Require: 2. Similarity threshold t .

Ensure: Insert Signature M into b hash tables $HT_1, HT_2, \dots, HT_b$

- 1: Compute values of b and r such that $t \approx (1/b)^{1/r}$
 - 2: // If $b = 16$ and $r = 4$, then the threshold is at $s = 1/2$ (4th root of 16 = 2)
 - 3: Instantiate b hash tables $HT_1, HT_2, \dots, HT_b$
 - 4: Partition M into b bands of r rows each.
 - 5: **for** each band $b_i; i \leftarrow 1$ to L/b **do**
 - 6: Vector `sign_chunk` $\leftarrow$ Get signature M 's chunk from band b_i
 - 7: Vector `rvec` $\leftarrow$ Get random vector of length r
 - 8: int `hashKey` $\leftarrow$ calculate dot product(`sign_chunk`, `rvec`)
 - 9: $HT_i.\text{INSERT}(\text{hashKey}, (M, \text{vector id}) \text{ pair})$
 - 10: **end for**
-

3.1 Collision probability using LSH scheme

Definition 3 (Candidate Probability). *For an LSH scheme working on top of MinHash signatures, the probability of becoming a candidate is given by P . Here r is the number of rows in a band and b is the number of bands (#signature partitions).*

$$P = 1 - (1 - s^r)^b. \quad (1)$$

Definition 4 (Calculating Threshold Similarity). *Using Definition 3, Threshold similarity is roughly the value of similarity s at which the probability of becoming a candidate is $1/2$. It is a function of b and r . The threshold is roughly where the rise is the steepest, and for large b and r we find that pairs with similarity above the threshold are very likely to become candidates, while those below the threshold are unlikely to become candidates. Here r is the number of rows in a band and b is the number of bands (#signature partitions).*

Example 1. $r=5, b=20$. Threshold $= (1/b)^{1/r} = 0.55$

Example 2. $r=10, b=10$. Threshold $= (1/b)^{1/r} = 0.79$

Example 3. $r=20, b=5$. Threshold $= (1/b)^{1/r} = 0.92$

$$\text{Threshold} = (1/b)^{1/r}. \quad (2)$$

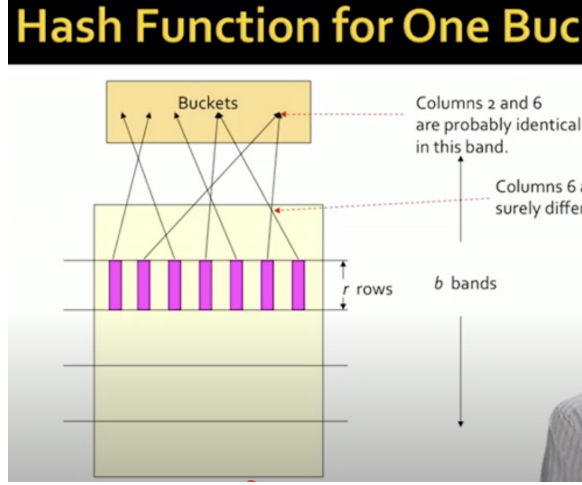


Figure 2: Showing hashing within a band of the signature matrix. If the signature chunks of two vectors are same within the band, then they will hash to the same hash table bucket. See Algorithm 1 for details. Image Credit: Youtube Video: “Lecture 14 — Locality Sensitive Hashing — Stanford University” by Prof. Jeffrey Ullman.

Definition 5 (Tuning r and b given user-defined Similarity Threshold t).

$$Threshold = (1/b)^{1/r}. \quad (3)$$

Here r is number of rows in a band and b is number of bands (#signature partitions). If Threshold similarity is provided by user, then we can find suitable values of b and r because $b \cdot r = \text{signature length} = L$. The two extreme cases are $b=1, r = L$
 $b=L, r = 1$

If we choose r to be a constant like 5, then $r = \text{constant}$, $b = L/r$

If $L = 20$, then the possible integer pairs (b,r) are $(1,20), (2,10), (4,5), (5,4), (10,2), (20,1)$. So, to find (b,r) pair that approximately equals to Threshold t , we can evaluate for all possible pairs and select the one that is close to t .
 If $t = 0.8$, then $(b,r) = (4,5)$ approximates t to 0.76.

3.2 Multi-Hash Query using a Feature vector’s Signature

The algorithm 2 shows the process of finding candidates via banded signature matrix partitioning.

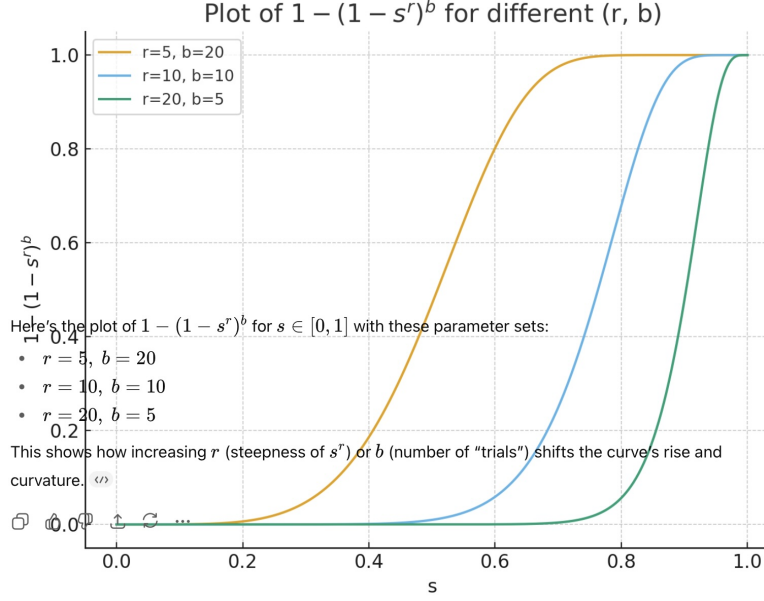


Figure 3: Y-axis is probability of becoming a candidate which is given by $1 - (1 - s^r)^b$. r = number of rows in a band and b is number of bands (#signature partitions).

X-axis shows the similarity values for arbitrary pairs of inputs. Given an arbitrary pair (a,b) with Jaccard similarity s , the formula $1 - (1 - s^r)^b$ gives the probability that the pair (a,b) will become a candidate pair.

Three plots 1) Yellow: $r=5, b=20$. 2) Blue: $r=10, b=10$. 3) Green: $r=20, b=5$. If similarity threshold $t=0.80$, then green curve is desirable over the yellow curve because for green curve, it very unlikely to have collision for pairs that are below the threshold. Therefore, using b and r controls the search for candidates.

- 1) Yellow: $r=5, b=20$. $t = (1/b)^{1/r} = 0.55$
- 2) Blue: $r=10, b=10$. $t = (1/b)^{1/r} = 0.79$
- 3) Green: $r=20, b=5$. $t = (1/b)^{1/r} = 0.92$

Algorithm 2 Find Candidates: Query a signature from b Hash Tables

Require: 1. Query Signature Vector $q = (q_1, q_2, \dots, q_L)$ of length L
Require: 2. Similarity threshold t .
Require: 3. b hash tables $HT_1, HT_2, \dots, HT_b$
Require: 4. Candidate Set C
Ensure: Output list of candidates with distances from query vector q .

- 1: Compute values of b and r such that $t \approx (1/b)^{1/r}$
- 2: Partition q into b bands of r rows each.
- 3: boolean isHit $\leftarrow$ false
- 4: **for** each band $b_i; i \leftarrow 1$ to L/b **do**
- 5: Vector sign_chunk $\leftarrow$ Get signature chunk from band b_i
- 6: Vector rvec $\leftarrow$ Get random vector for b_i
- 7: // rvec should be consistent for both insertion and query
- 8: int hashKey $\leftarrow$ calculate dot product(sign_chunk, rvec)
- 9: $HT_i \leftarrow$ Get hash table for band b_i
- 10: boolean isFound $\leftarrow$ If sign_chunk in HT_i by $HT_i.QUERY(hashKey)$
- 11: **if** isFound == true **then**
- 12: isHit $\leftarrow$ isHit $\vee$ isFound // $\vee$ is logical OR
- 13: retrieve the (signature, feature vector id) pair from HT_i and add it to candidate set C
- 14: **end if**
- 15: **end for**
- 16: **for** each candidate c in the candidate set C **do**
- 17: float ss $\leftarrow$ Find signature similarity(q, c)
- 18: **if** ss $\geq t$ **then**
- 19: add (ss, vector id) pair to the output list
- 20: **end if**
- 21: **end for**
- 22: **return** sorted output candidate list for query q .
- 23: *Check:* Using set to insert candidate signatures should take care of duplicate signatures because there may be hash collisions in multiple bands/hashtables HT_i .

s	$1 - (1 - s^r)^b$
.2	.006
.3	.047
.4	.186
.5	.470
.6	.802
.7	.975
.8	.9996

Values of the S-curve for $b = 20$ and $r = 5$

.. The probability that the signatures agree in all the rows of at least one band, and therefore become a candidate pair, is $1 - (1 - s^r)^b$.

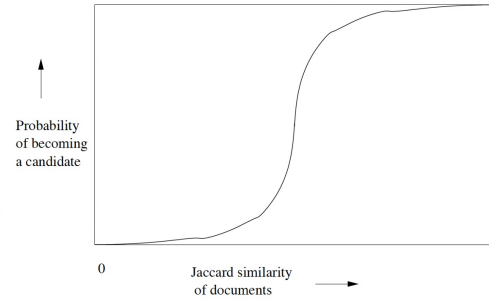


Figure 3.8: The S-curve

Figure 4: s = similarity. b = #bands. r = #rows. If we follow the hashing algorithm described earlier via Signature Matrix partitioning, given two inputs (a,b) with Jaccard similarity s , the formula $1 - (1 - s^r)^b$ gives the probability that the pair (a,b) will become a candidate pair. This is because both a and b will get inserted into the same hash table bucket with the stated probability. The right figure is drawn using the data from the left table.